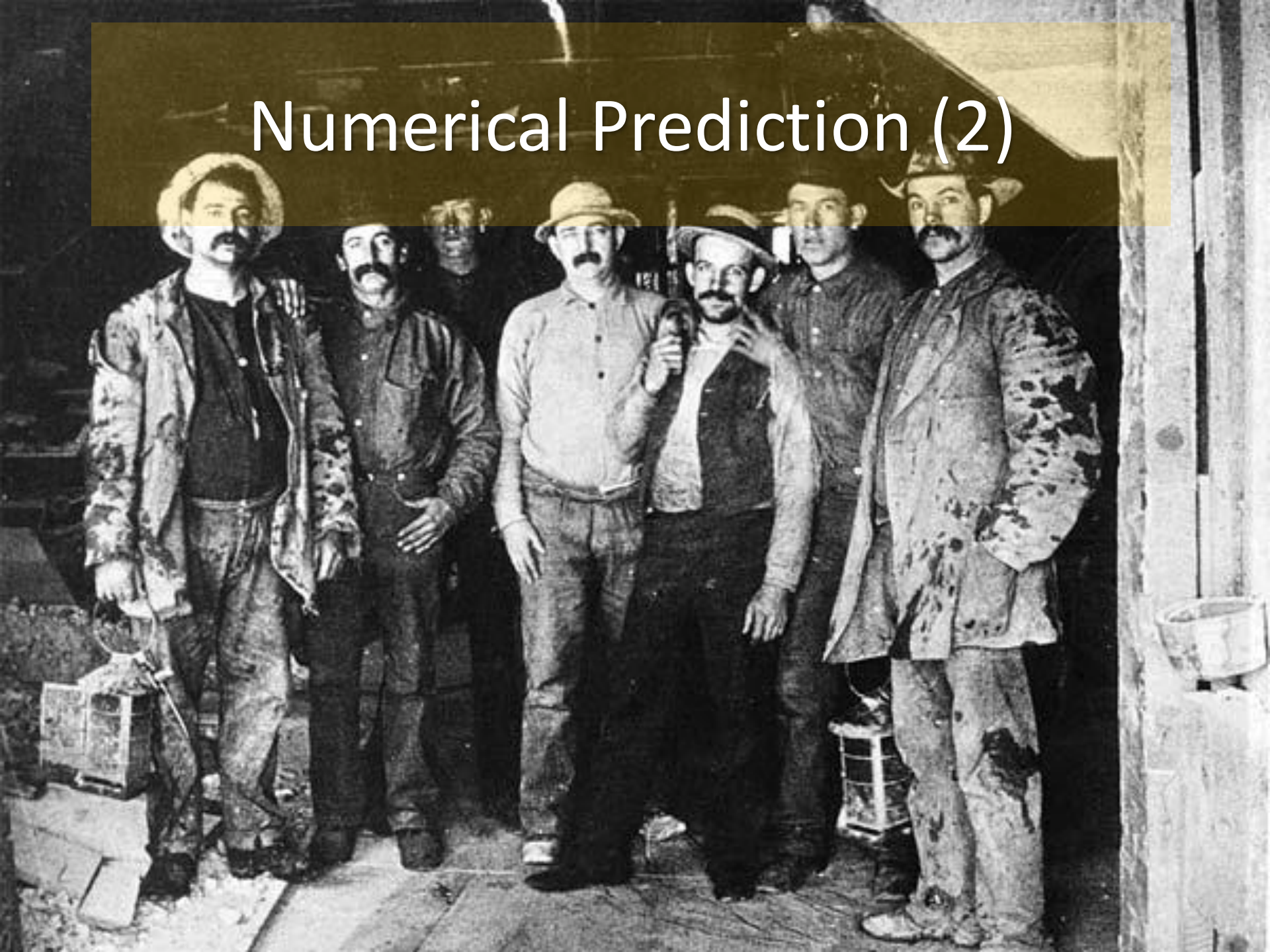
Numerical Prediction (2)



Some organisation

- Interviews have now been scheduled

- You are responsible for ensuring the interview takes place
- Can still move (but limited spots)
- Stay with your own TA

Limitation of linear models

- non-numeric attributes
- linear dependency might be only “local”

- **Example:** **predicting rise time of a servo system**
- **Task:** Predict the rise time of a servomechanism in terms of two (continuous) gain settings and two (discrete) choices of mechanical linkages
  - an extremely non-linear phenomenon
  - see `servo.arff` online

Servo data sample

motor	screw	pgain	vgain	time
E	E	5	4	0.281251
B	D	6	5	0.506252
D	D	4	3	0.356251
B	A	3	2	550.003
D	B	6	5	0.356251
E	C	4	3	0.806255
C	A	3	2	510.001
A	A	3	2	570.004
C	A	6	5	0.768754

C. Regression tree/model trees

- Main idea:
  - Use decision trees to identify “linear regions”
  - Find linear model for each region
- ***Detail 1.***

  While splitting a node use “standard deviation reduction”

$$SDR = sd(T) - \sum_i \frac{|T_i|}{|T|} \times sd(T_i)$$

From page 243

Details continued

- ***Detail 2.***
  - stop criterion: node covers less than *k* cases or std reduction is smaller than *p* percent (e.g. *k*=10, *p*=5)
- ***Detail 3.*** in the leaf node, use
  - either "the mean value" → regression tree
  - or a "linear regression model" → model tree
- ***Detail 4.***
  - to avoid overfitting reduce the size of the tree by pruning
- ***Detail 5.***
  - to avoid overfitting "average predictions" using smoothing

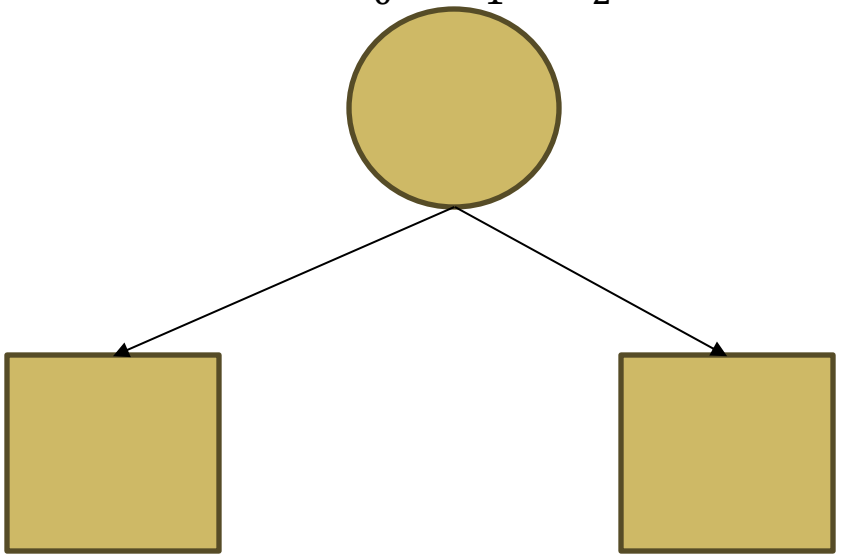
Pruning

- Main idea:
  - to avoid overfitting remove some subtrees to make the tree smaller
- Details
  - for every node, find the “simplest” linear model (backward selection heuristic) and estimate its error
  - for every node calculate the weighted average of errors of child nodes
  - remove all the children when appropriate

Pruning example

$LM_0 = w_0 + w_1 x_1 + \cdots + x_v$

$n_0 = n_1 + n_2$ cases



n_1 cases

n_2 cases

$LM_1 = w_0 + w_1 x_1 + \cdots + x_v$ $LM_2 = w_0 + w_1 x_1 + \cdots + x_v$

could use different attributes *v*!

was this split a good idea?

estimate generalisability to test set

$error_{test} = error_{train} \cdot (n+v)/(n-v)$,

compare estimated test errors of LM1 and LM2 with LM0

Smoothing

- **Main idea:** to avoid too detailed splits "propagate" predicted values from the leaf to the root using a "smoothing formula":

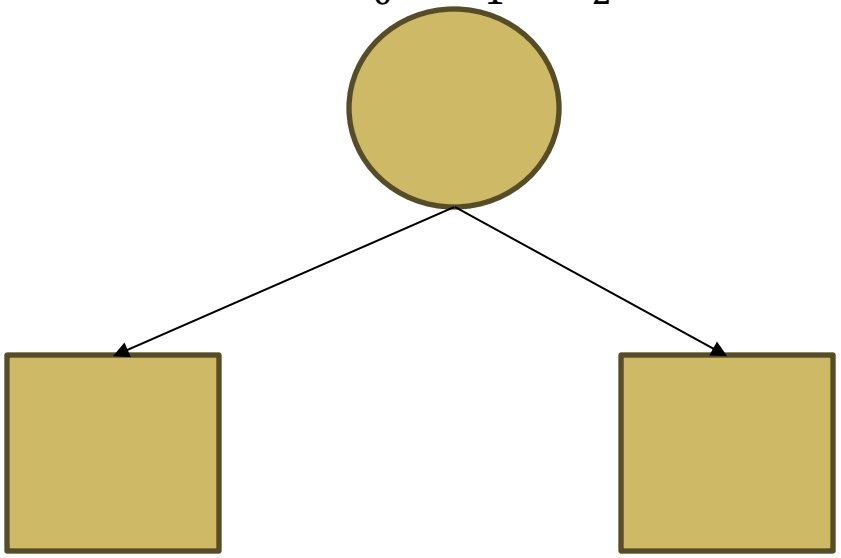
$$p_{prop} = \frac{np+kq}{n+k}$$

  - p_{prop} – propagated value
  - p – predicted value from the child
  - q – predicted value from the node
  - n – number of cases covered by the child node
  - k – smoothing factor

Smoothing example

$LM_0 = w_0 + w_1 x_1 + \cdots + x_v$

$n_0 = n_1 + n_2$ cases



n_1 cases

n_2 cases

$LM_1 = w_0 + w_1 x_1 + \cdots + x_v$

$LM_2 = w_0 + w_1 x_1 + \cdots + x_v$

Is LM1 overly specific?

$$p_{prop} = \frac{n_1 p + kq}{n_1 + k}$$

LM_1 predicts p
LM_0 predicts q
smoothing factor k

Nominal attributes

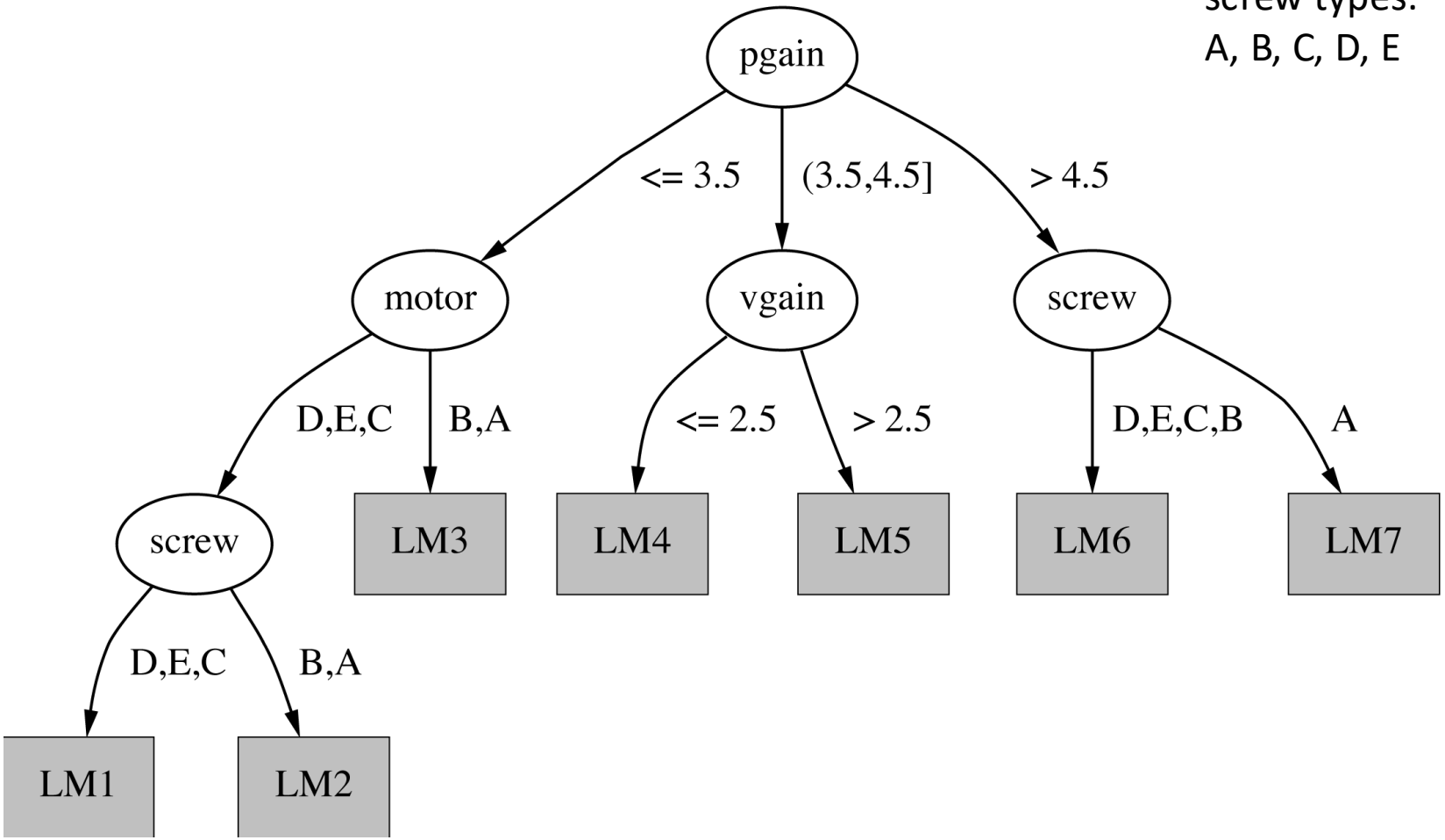
- What if an attribute has many values?
- A nominal attribute with *k* values
  can be replaced by *k-1* binary attributes:
  - for every value of an attribute, find the corresponding mean of the class value
  - sort values according to means
  - consider *k-1* possible splitting points as new attributes

(this procedure has to be repeated at every node !)

Means change!

Servo data example

screw types:
A, B, C, D, E



Servo linear models

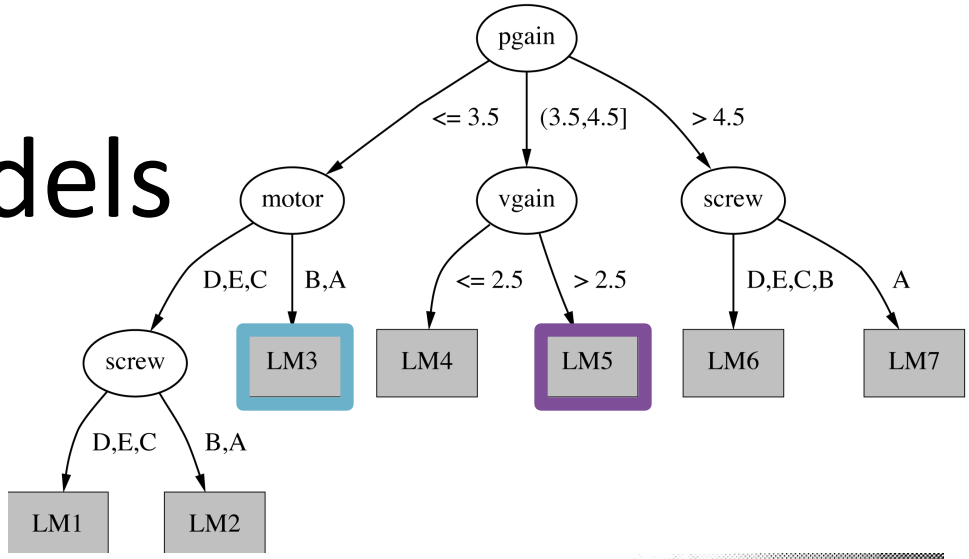


Table 6.1 Linear models in the model tree.

Model		LM1	LM2	LM3	LM4	LM5	LM6	LM7
Constant term		–0.44	2.60	3.50	0.18	0.52	0.36	0.23
pgain								
vgain		0.82		0.42				0.06
motor = D	vs. E, C, B, A		3.30		0.24	0.42		
motor = D, E	vs. C, B, A	1.80			–0.16		0.15	0.22
motor = D, E, C	vs. B, A				0.10	0.09		0.07
motor = D, E, C, B	vs. A			0.18				
screw = D	vs. E, C, B, A							
screw = D, E	vs. C, B, A	0.47						
screw = D, E, C	vs. B, A	0.63		0.28	0.34			
screw = D, E, C, B	vs. A			0.90	0.16	0.14		

Locally weighted regression

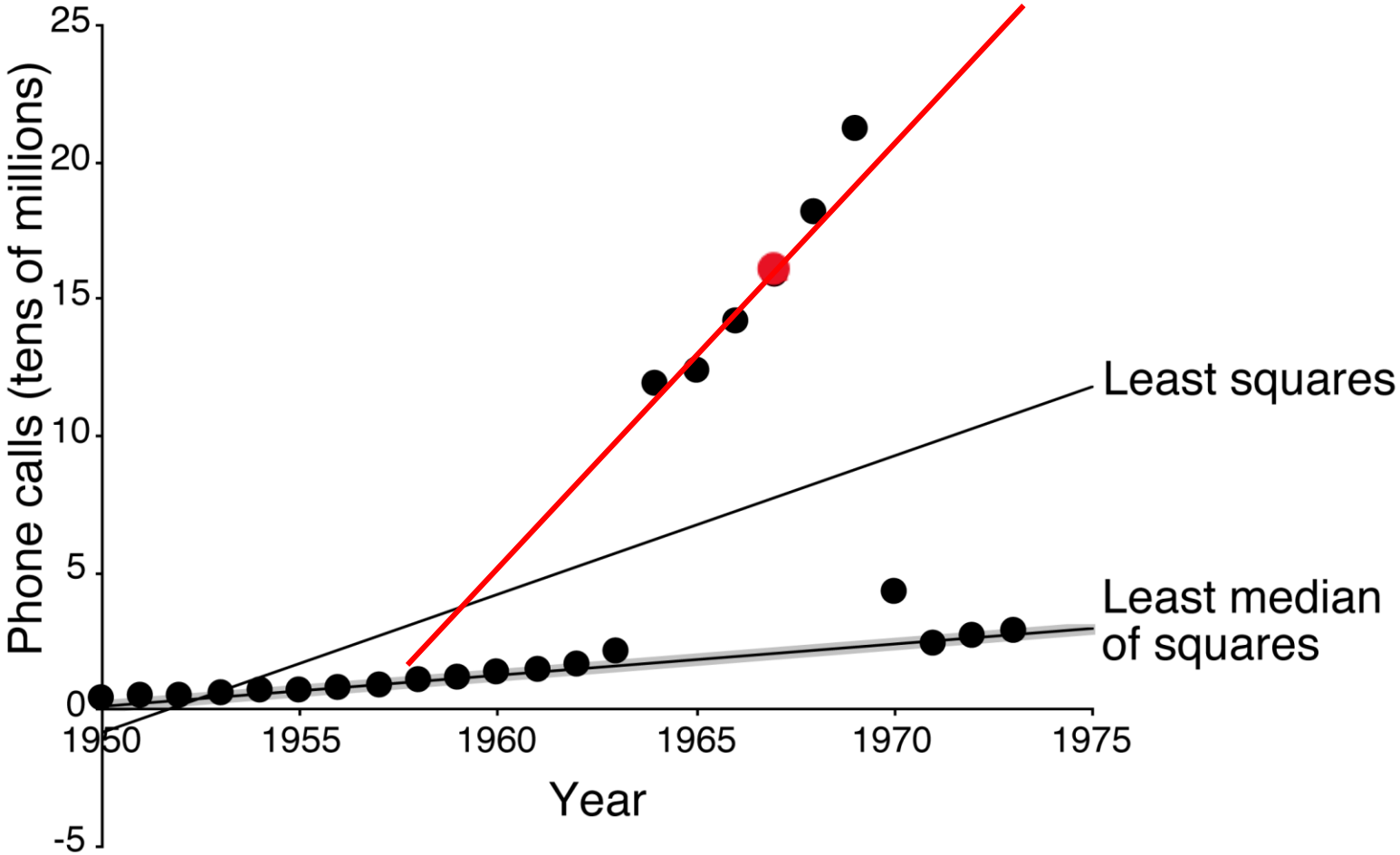
Main idea to approximate f(x):

- look only at points that are close to x;
- build a linear model for these points
- apply the model to x

Details:

1. Given x, define weights of all points in the training set:
   - $w_i = r(|x-x_i|)$; – radial basis function, e.g. Gaussian pdf
   - weights should reflect the distance to "known" points (the closer the point the higher the weight)
2. Build a model for the weighted linear regression problem:
   - $weighted\ error = w_1 (a_1-y_1)^2 + w_2 (a_2-y_2)^2 + \ldots + w_k (a_k-y_k)^2$

Belgium telephone calls again..

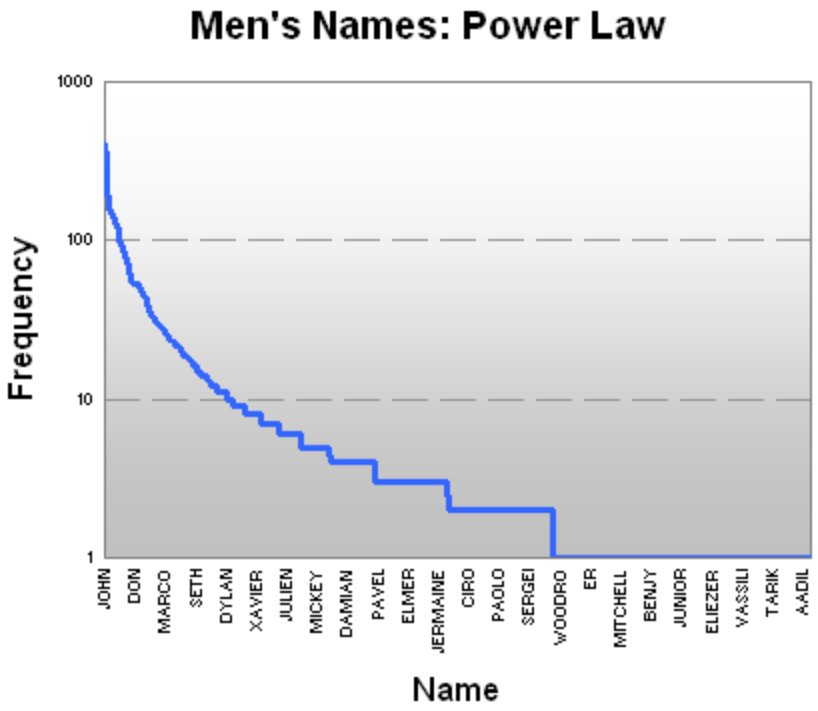


Transforming data so linear regression can work

- Setup:
  - Output is not a linear combination of input

    $y = ax+b$ gives big errors ($x \in R^n$, n often larger than 1)

    Plotting x and y reveals something else than a line/plane



This does not look like a plane cutting the 3D space into two

Idea 1

- x and y might be transformed so you can build linear models on the transformed attribute

- Example:
  - Relationship looks exponential: $y = ae^{bx}$
  - Take the logarithm of both sides: $\ln(y) = \ln(a) + bx$
  - Now you can build a **linear** model

Idea 2

- If you think it makes more sense, perhaps transform x only to a suitable format
  - Example: x_1 and x_2 seem not to be independent
  - Does the transformation $x_{new} = a_1x_1 + a_2x_2$ make sense?
  - Does it make more sense to, e.g. have $x_1 * x_2$ or $\mathbf{x_1 * x_2^2}$? Or $x_1 * \ln(x_2) * x_2$, etc.?
  - Linear model will look like: $y = a_1x_1 + a_2x_2 + a_3x_1x_2$

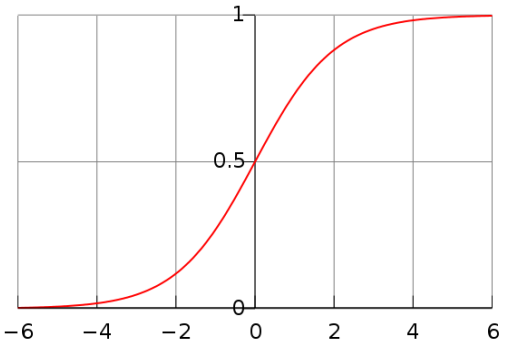
for more inspiration on feature engineering, see e.g. python packages
- feature-engine
- mlxtend
- sklearn.preprocessing

GLMs

- To learn more, look up generalised linear models (GLM)
  - There are a number of functions you can use to turn something into a linear regression problem
  - Logistic regression is a GLM
    - It builds a linear model for a binary output (class)

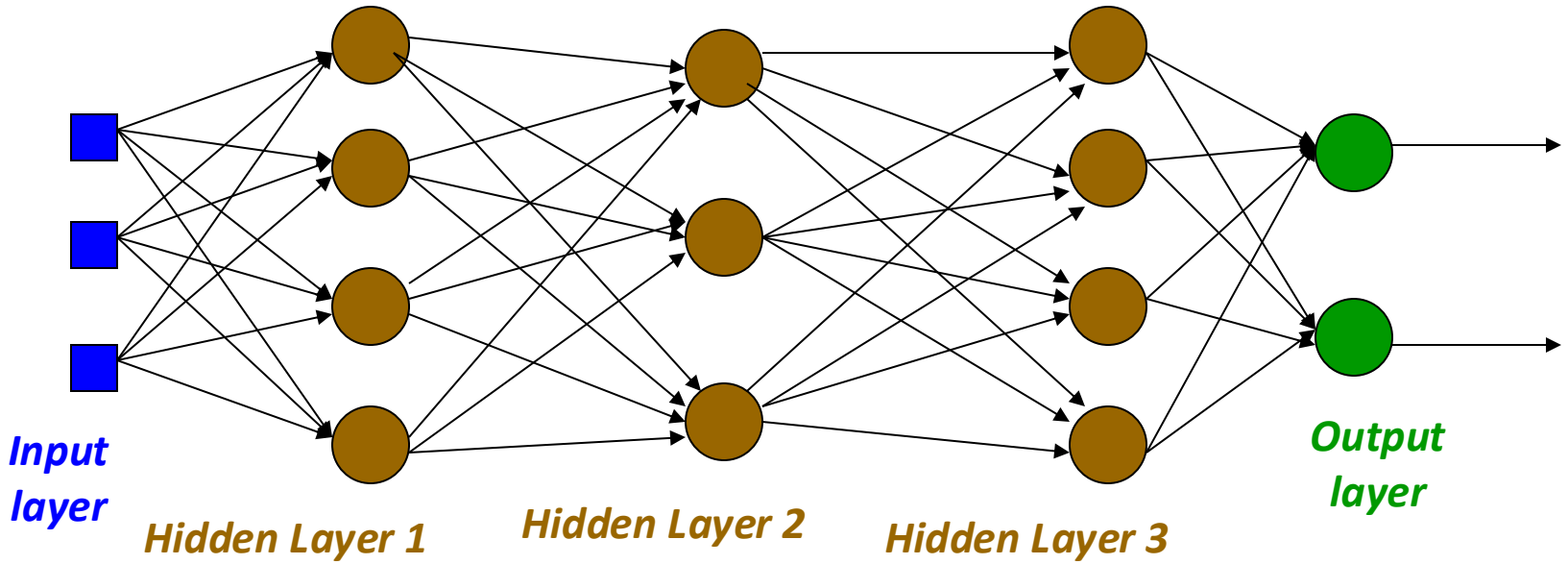
$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \cdots + \beta_k x_k,$$

$$f(z) = \frac{e^z}{e^z + 1} = \frac{1}{1 + e^{-z}}$$



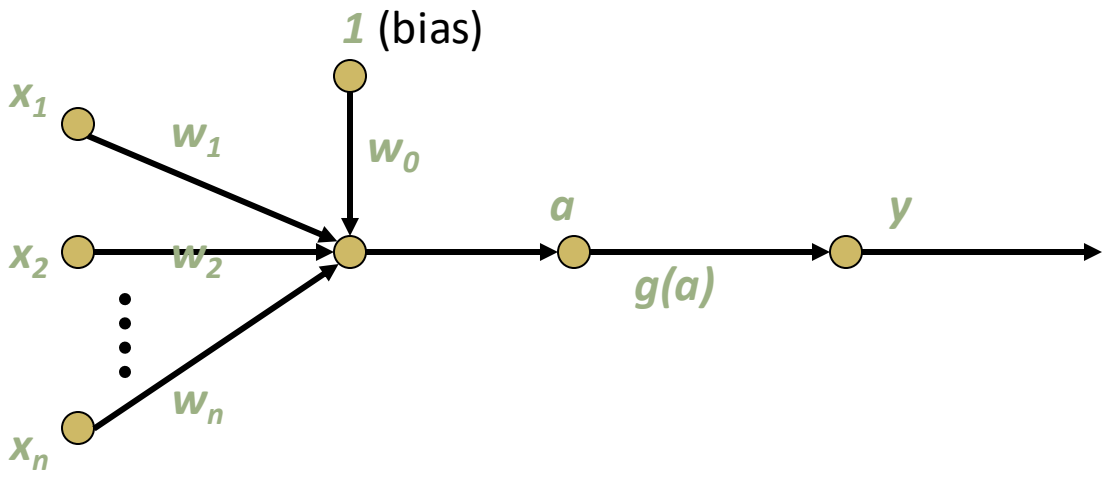
Neural Networks (continued)

- Neural networks can also be used for regression
- Remember the earlier lecture



Neural Networks

$$a = w_0 + \sum w_i x_i; \qquad j(a) = \begin{cases} +1 \text{ if } a > 0 \\ 0 \text{ if } a = 0 \\ -1 \text{ if } a < 0 \end{cases}$$

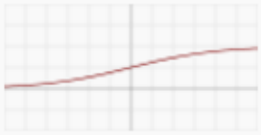
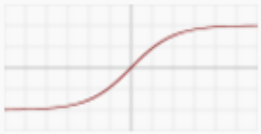
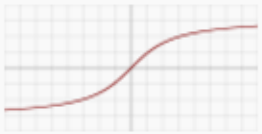


Neural Networks

- So far we did not really go into detail on the activation function
  - We only looked at a simple threshold function (slide before)
  - Clearly this function is not suitable for regression
  - We did assume a differentiable function in the back propagation
  - So what functions are there?

Activation functions

- Let us consider some of the most common ones:

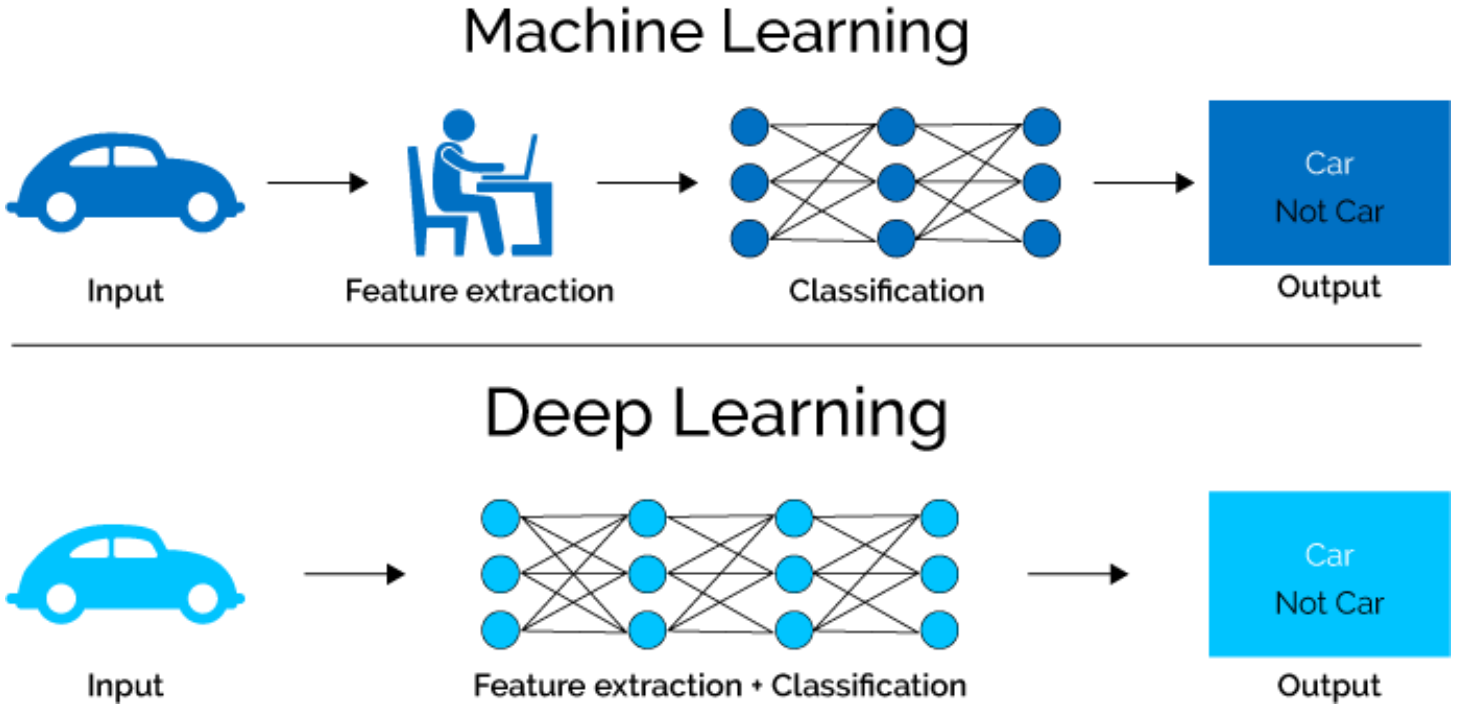
Name	Plot	Equation	Derivative
Identity		$f(x) = x$	$f'(x) = 1$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
Logistic (a.k.a Soft step)		$f(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1-f(x))$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Activation functions

- Normally you use one type of activation function per layer
- Type of problem determines the activation function of the output layer

Deep learning

- We can make networks more and more complex
- This allows the networks to take care of some work for use



https://www.xenonstack.com/blog/static/public/uploads/media/machine-learning-vs-deep-learning.png

Deep learning

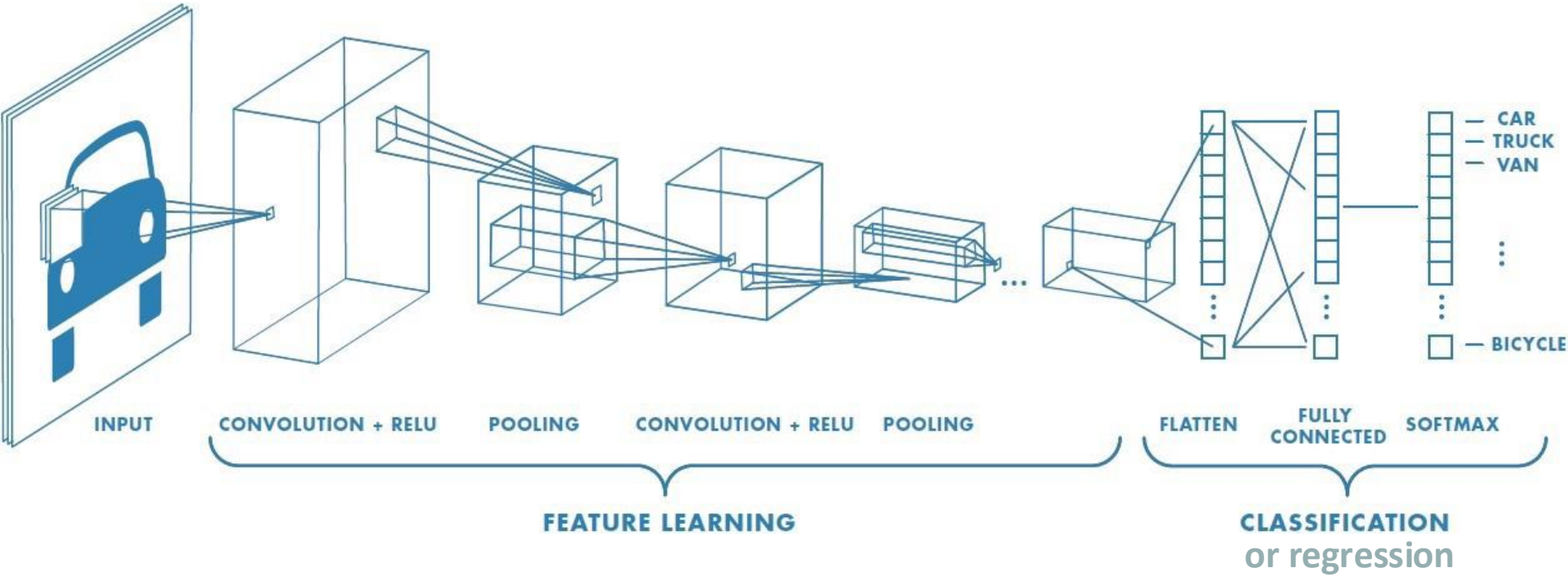
- Want to discuss two types of deep learning (briefly):
  - Convolutional neural networks
    - Can be used for a variety of purposes, mainly originates from the image recognition field
  - Long Short Term Memory Networks
    - Can be used for temporal data

Convolutional Neural Networks

- We introduce different types of layers to extract features from our data
  - Convolutional layers, pooling layers
- In the end, we feed the output from these layers into a “regular” feed forward multi-layer perceptron

Convolutional Neural Networks

- An example:



https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53

Convolutional Neural Networks

- Convolutional layers

1	2	1	1
0	1	0	2
0	1	0	1
1	1	1	0

Data

x

1	0
0	1

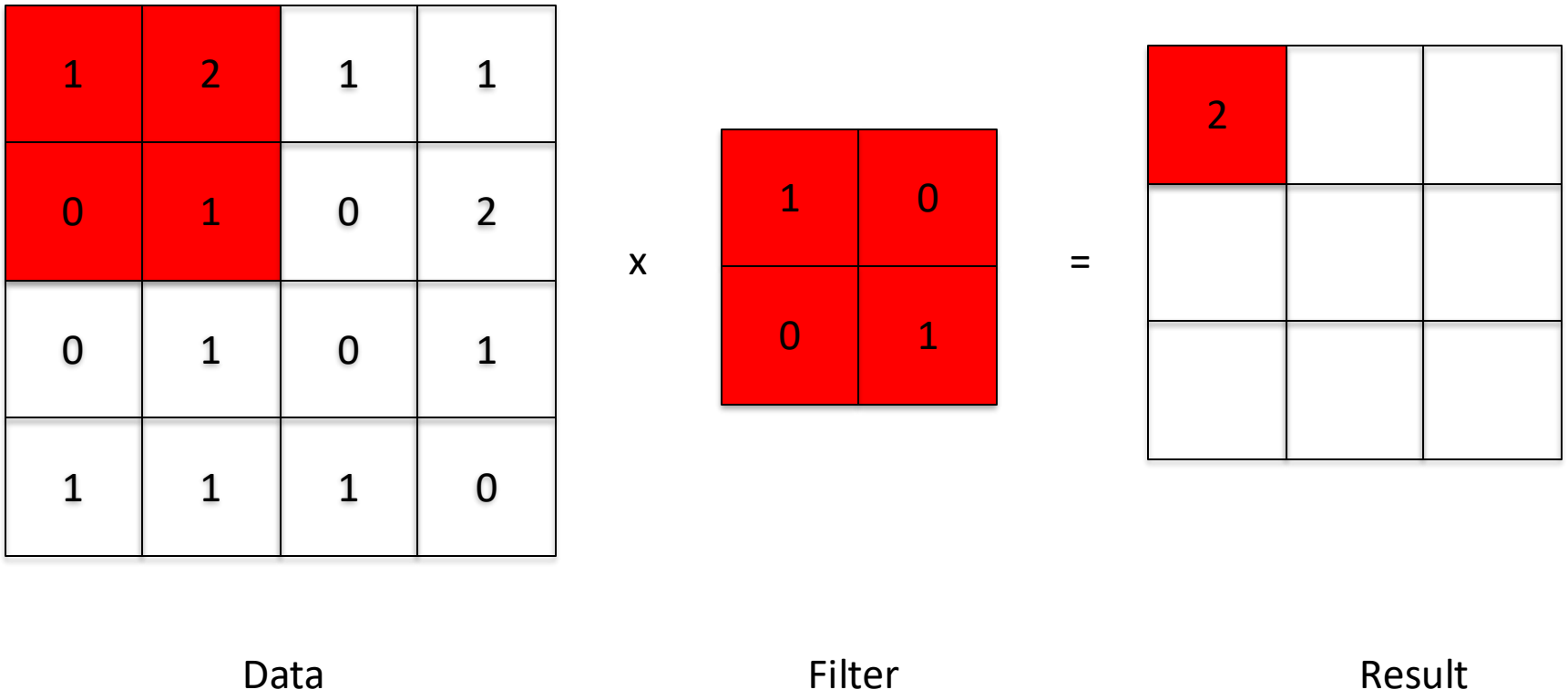
Filter

=

Result

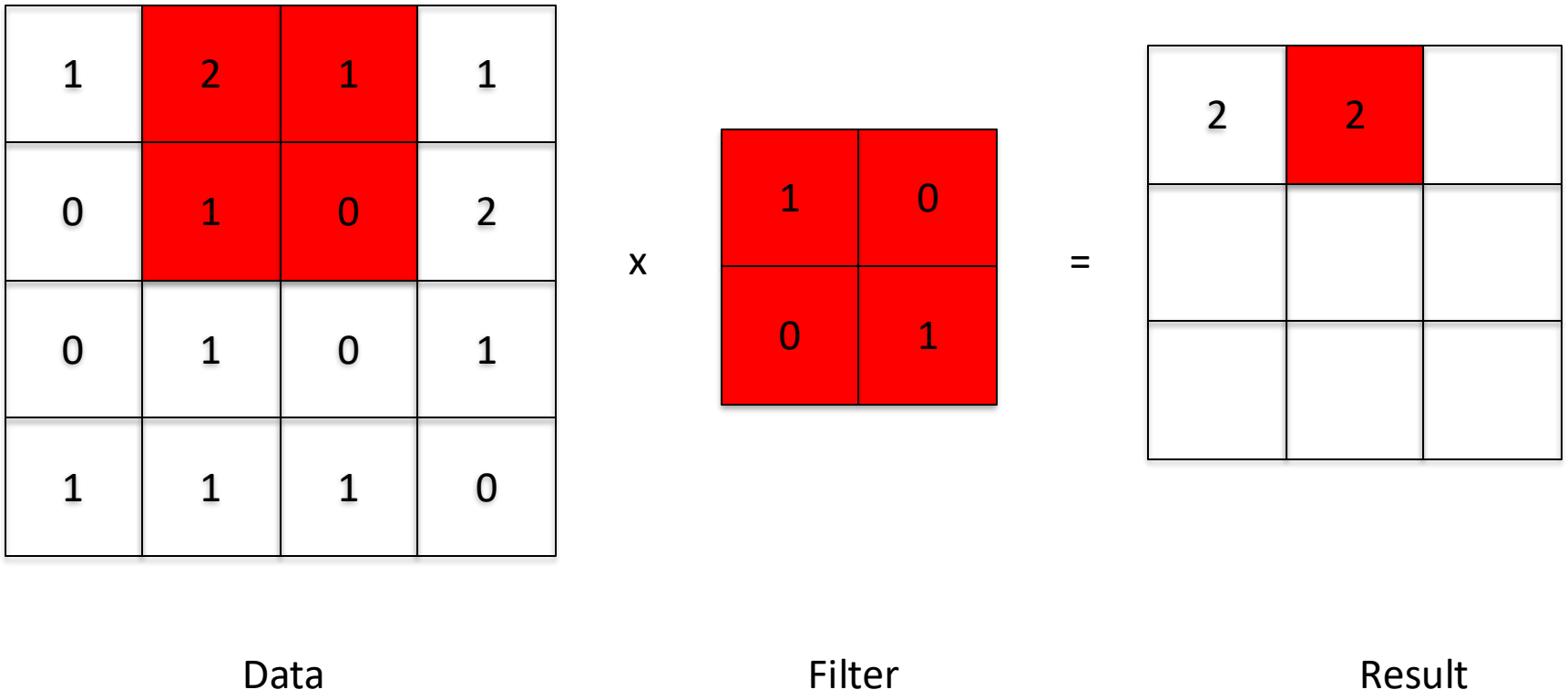
Convolutional Neural Networks

- Convolutional layers



Convolutional Neural Networks

- Convolutional layers



Convolutional Neural Networks

- Can learn weights of filter based on back propagation
- Layers contain a lot less weights than fully connected layers
- How much do we move to the side?
  - Assumed 1 now, can vary (e.g. 2)
  - Called the stride
- We reduce the dimensions now:
  - We might want a lot of layers
  - The corners could be interesting
  - Solution: add zeros to the side of the data to maintain the dimensions (zero padding)

Convolutional Neural Networks

- Pooling layers:
  - Down sample your data
  - Max pooling with filter size 2 x 2 and stride 2

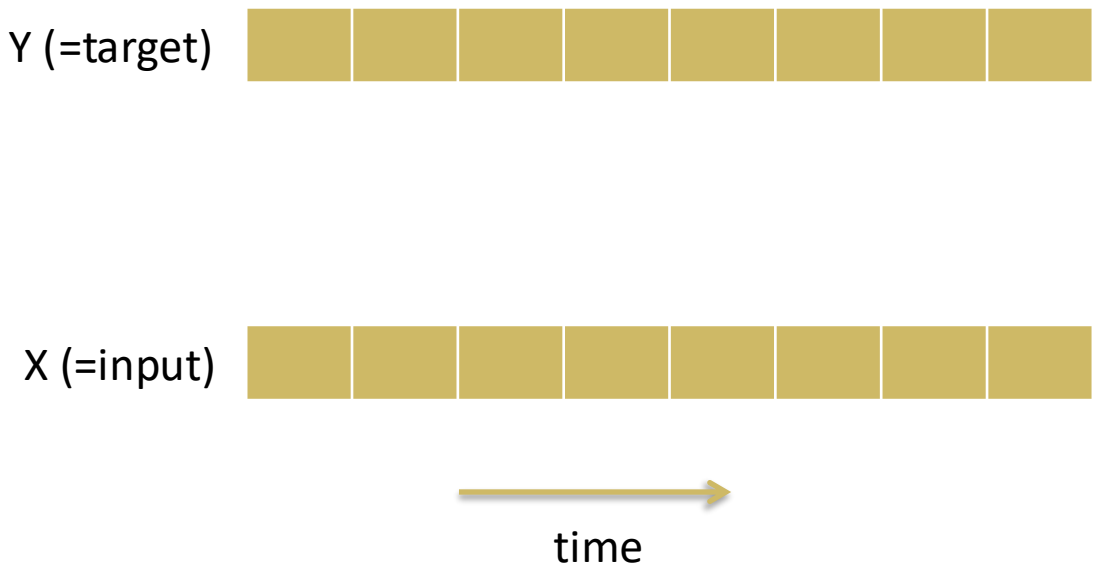
1	2	1	1
0	1	0	2
0	1	0	1
1	1	1	0

=>

2	

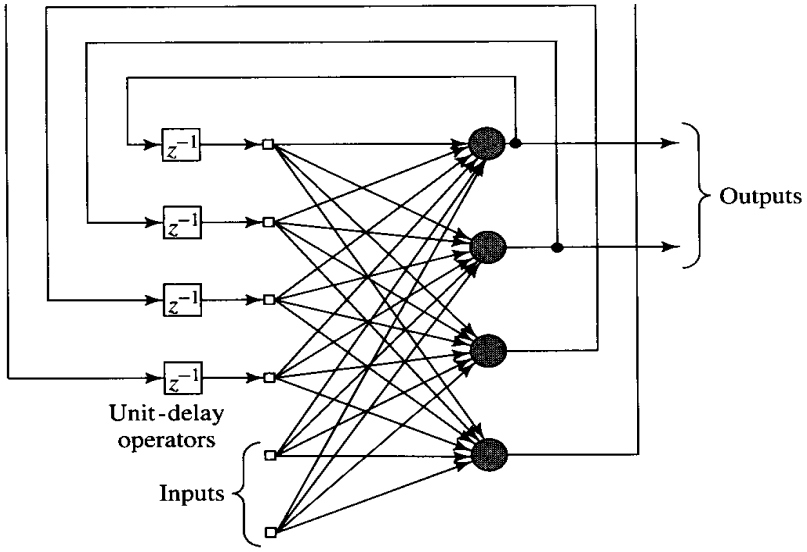
Recurrent networks

- What if you want to predict a sequence of values?



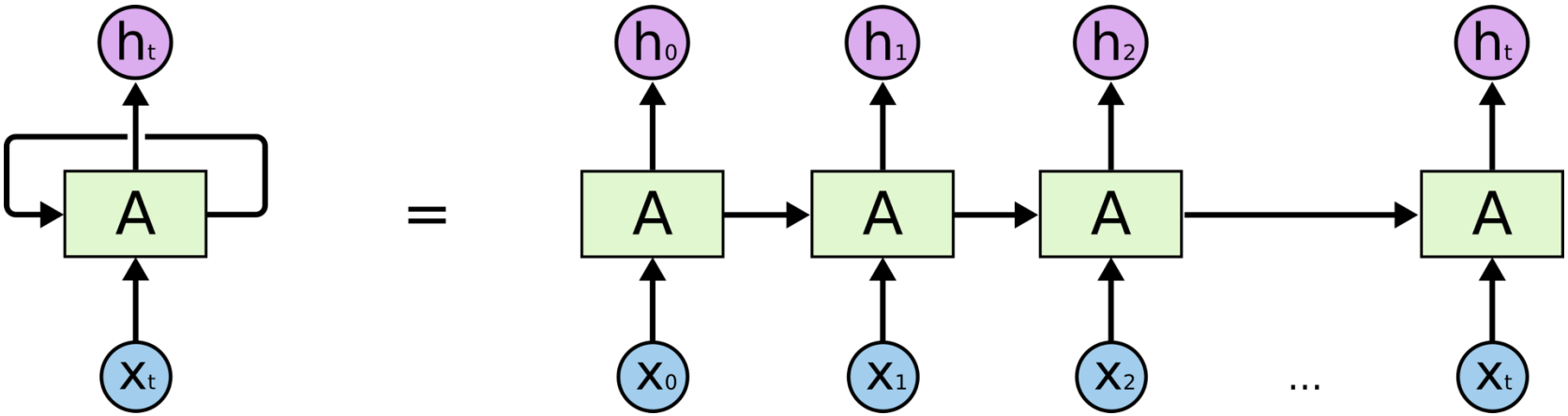
Recurrent networks

- Remember these was among the variants we have discussed before



Recurrent networks

- How do we train them?
- Can we just use back propagation?
- Nope, we also need to blame our previous prediction (back propagation through time)



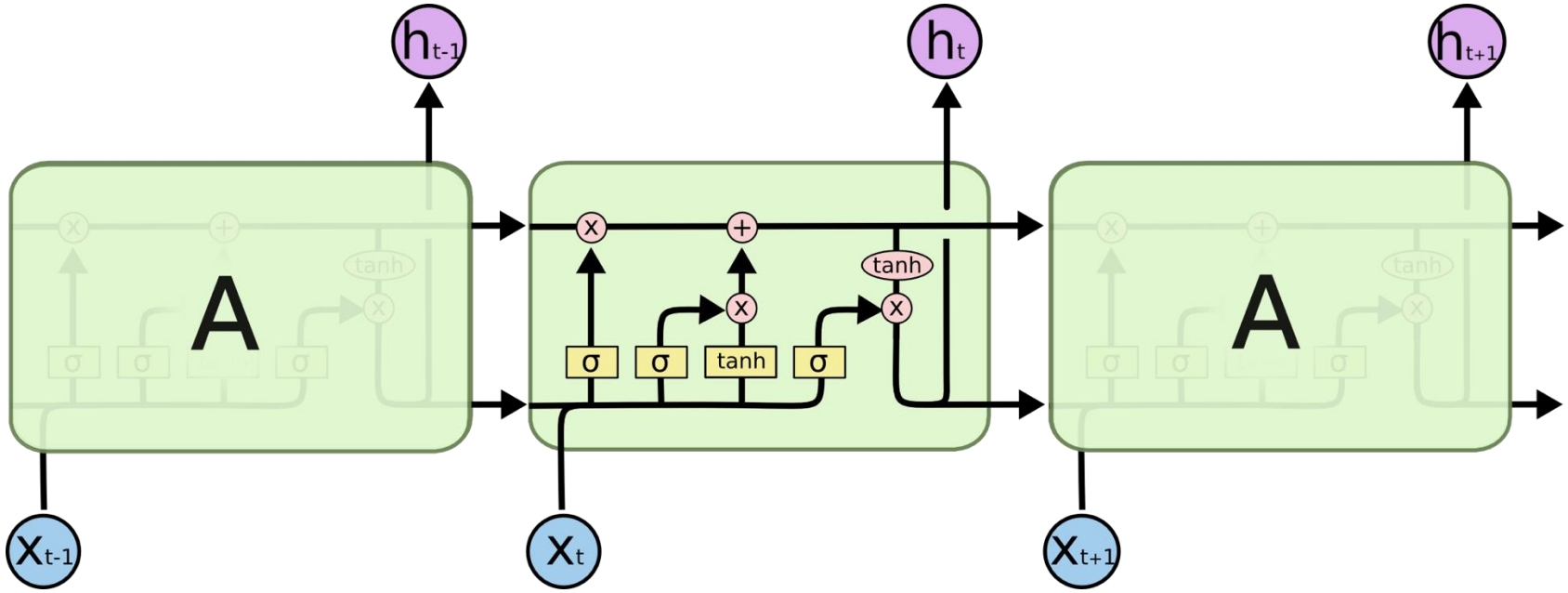
Images from https://colah.github.io

LSTM

- Would it be easy to learn over time?
- Nope, vanishing gradient problem, difficult to learn long term dependencies
- Therefore LSTM's (Long Short Term Memory networks) have been proposed (Hochreiter and Schmidhuber, 1997)
- Life does get quite complex……

LSTM

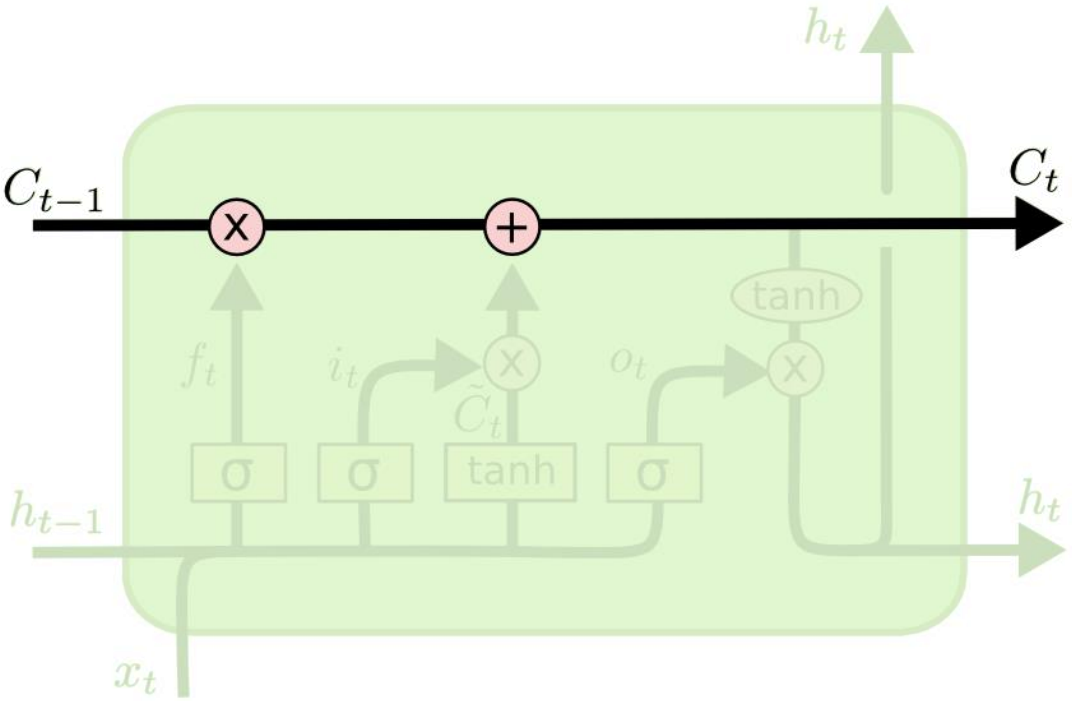
- The general idea: more complex internal structure of units



Images from https://colah.github.io

LSTM

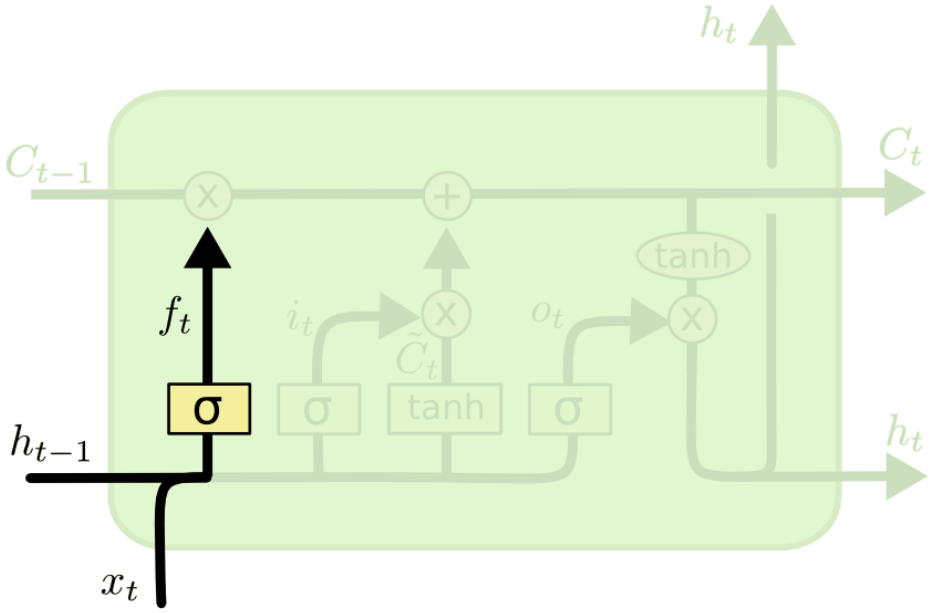
- Walk through of the different parts
- We have an internal cell state *C* (a vector of numbers) that we update based on gates



Images from https://colah.github.io

LSTM

- The first gate: the forget gate
- Determines for each part of the cell state (i.e. each element in the vector) how much to keep

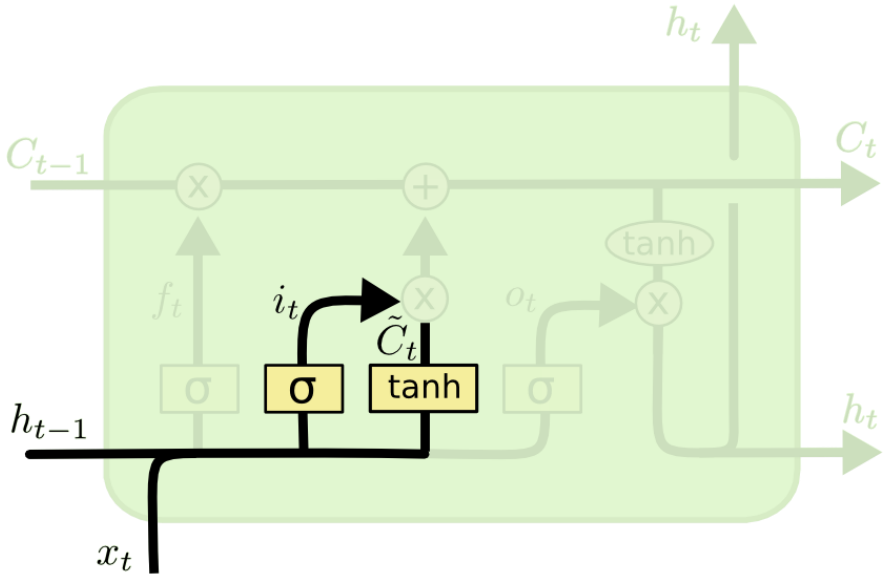


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Images from https://colah.github.io

LSTM

- Next: we decide on what to add to our cell state:
  - Which part to update (i_t)
  - How to update ($\tilde{C}_t$)



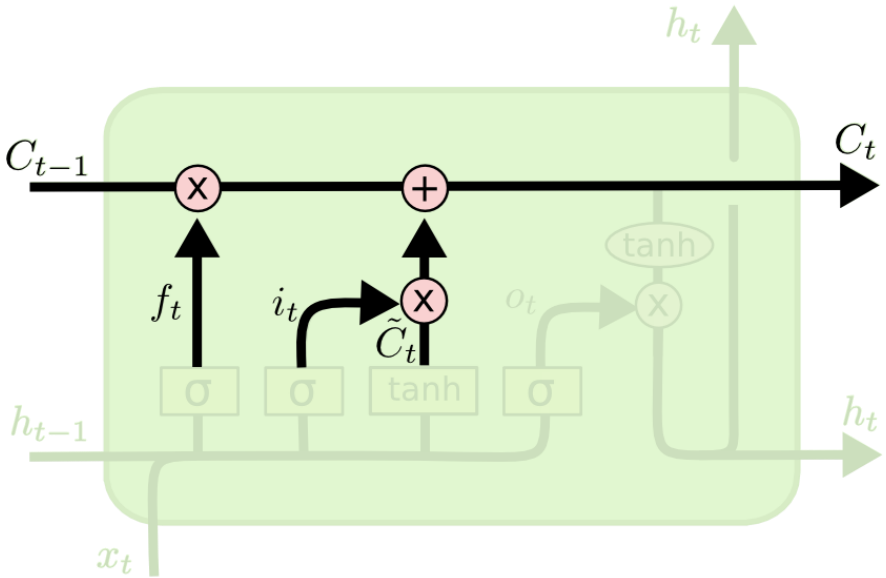
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Images from https://colah.github.io

LSTM

- And we update the cell state based on what to forget and what we want to add to the cell state

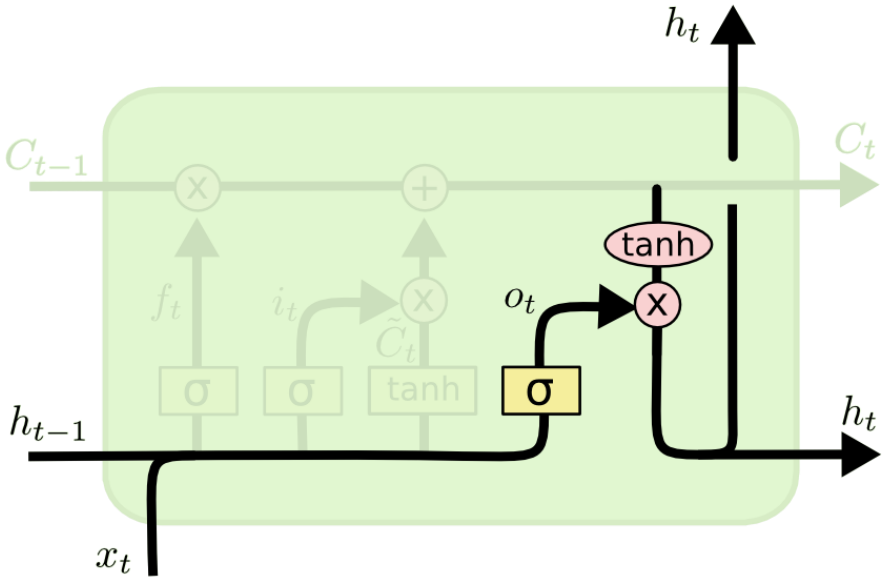


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Images from https://colah.github.io

LSTM

- Finally, we decide on the output based on our previous prediction, the cell state and our current input



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Images from https://colah.github.io

Summary: non-linear numerical prediction

- Regression trees & models
- Locally weighted regression
- GLMs
- Deep learning
  - Convolutional neural networks
  - LSTM